

INF2007 – TN5 – Melissa Moya

Architecture concurrente

Un segment découpé à exactement N caractères risque de couper un mot en deux, faussant le compte. Pour éviter ce problème, *splitIntoSegments* avance la coupure caractère par caractère jusqu'au prochain espace blanc, garantissant que chaque segment contient des mots complets. Cette décision architecturale est préalable à toute concurrence, car elle assure que chaque goroutine peut compter ses mots de façon totalement indépendante sans coordination.

Une fois les segments établis, une goroutine est lancée par segment avec *go countWordsInSegment*. Chaque goroutine calcule un total local dans une variable de pile, puis envoie ce résultat sur un canal partagé (*chan int*).

Trois propriétés du canal et du programme garantissent l'exactitude du résultat sans recourir à un mutex. Premièrement, aucune variable n'est partagée entre les goroutines, ce qui élimine toute condition de course. Deuxièmement, le canal est en mémoire tampon avec une capacité égale au nombre de segments, de sorte que chaque envoi aboutit immédiatement sans bloquer la goroutine expéditrice, quelle que soit la vitesse de la goroutine principale. Troisièmement, la boucle *for range segments* consomme exactement N résultats avant de retourner, ce qui constitue un mécanisme de synchronisation implicite entre les goroutines. L'addition étant commutative, l'ordre d'arrivée des résultats n'affecte pas le total final. Ce comportement correspond au modèle de Go basé sur la communication entre goroutines via des canaux pour assurer une exécution concurrente correcte.

Démarche de mesure et résultats

Les mesures ont été effectuées sur un Intel i5-10300H à 2,50 GHz (4 cœurs physiques et 8 threads logiques, Windows/amd64) avec un contenu de test de 100 000 mots (~700 000 caractères). Chaque configuration a été exécutée 6 fois avec *go test -bench=.* *-count=6* et analysée avec *benchstat* pour obtenir la médiane et l'intervalle de confiance à 95 %. Le contenu de test est pré-généré comme variable de paquet, hors de la boucle *b.N*, ce qui exclut l'initialisation du temps mesuré.

Tableau 1 – Temps selon la taille des segments (CountWordsConcurrent)

Segment	Séq.	500	1 000	5 000	10 000	50 000	100 000	Tout
Temps (ms)	5.08	8.87	2.84	4.84	4.85	1.78	1.97	4.09

Le résultat révèle un comportement en U. Avec des segments trop petits (500 caractères, soit environ 1 200 goroutines), le coût de création et d'ordonnancement des goroutines dépasse le bénéfice du parallélisme, dégradant la performance en deçà du séquentiel. À 50 000 caractères on atteint l'optimum à 1.78 ms, soit un gain de 2.85× sur le séquentiel. Au-delà, les segments deviennent si grands que le parallélisme disparaît et on retrouve un comportement quasi-séquentiel.

Analyse de la linéarité et worker pool

Pour répondre à la question de la linéarité, *CountWordsConcurrentN* contrôle directement le nombre de goroutines en calculant une taille de segment proportionnelle.

Tableau 2 – Temps selon le nombre de goroutines (*CountWordsConcurrentN*)

Workers	1	2	4	8	16	32
Temps (ms)	10.39	9.02	8.47	6.93	6.31	6.21
Accélération	1.00×	1.15×	1.23×	1.50×	1.65×	1.67×

La progression est clairement sous-linéaire. Sur 4 cœurs physiques, l'accélération plafonne à 1.67× au lieu des 4× théoriques. En effet, *strings.Fields* effectue un seul balayage linéaire très rapide, si bien que le temps de calcul par goroutine est trop court pour amortir les coûts fixes de création, d'ordonnancement et de synchronisation via le canal. La lecture du fichier, le découpage et la synchronisation constituent une part séquentielle incompressible limitant l'accélération. Face à ce constat, *CountWordsConcurrentPool* adopte une approche différente. Plutôt que de créer une nouvelle goroutine par segment à chaque appel, cette fonction maintient un pool fixe de *numWorkers* goroutines persistantes qui lisent les segments depuis un canal de jobs via *for seg := range jobs*. La fermeture du canal avec *close(jobs)* signale proprement la fin du travail à tous les workers simultanément, sans coordination explicite supplémentaire. L'utilisation de deux canaux séparés, soit un canal *jobs* pour distribuer les segments aux workers et un canal *results* pour collecter les comptes partiels, reflète le principe fondamental selon lequel les canaux servent à la fois à la communication et à la synchronisation entre goroutines sans partage de mémoire. Cette architecture élimine la recréation répétée des goroutines et fixe la taille des segments à 50 000 caractères, soit l'optimum identifié au Tableau 1.

Tableau 3 – Worker pool vs goroutine-par-segment

Workers	1	2	4	8	16	32
Pool (ms)	2.86	2.08	1.71	1.59	1.46	1.53
Par-segment (ms)	10.39	9.02	8.47	6.93	6.31	6.21
Gain pool	3.6×	4.3×	5.0×	4.4×	4.3×	4.1×

Le pool est 4 à 5× plus rapide que l'approche par segment à nombre de workers égal. La différence vient du fait que *CountWordsConcurrentN* avec 1 worker produit un seul segment de 700 000 caractères, ce qui crée un comportement quasi-séquentiel, tandis que le pool fixe toujours la taille à 50 000 caractères, soit environ 14 segments distribués au worker unique. Le pool atteint 1.96× d'accélération de 1 à 16 workers, contre 1.67× pour l'approche naïve, car la réutilisation des goroutines réduit la part séquentielle d'orchestration. Le plateau dès 16 workers persiste, la bande passante mémoire constituant un plafond indépendant de la stratégie choisie.

En définitive, le modèle de concurrence de Go, basé sur des goroutines légères et des canaux, permet d'atteindre une accélération réelle sur du matériel multicœur, à condition d'adapter la granularité des tâches et de réutiliser les goroutines afin de minimiser les coûts fixes d'orchestration.